SIMATS ENGINEERING

CSA0705 - COMPUTER NETWORKS

STUDENT NAME : BALAJI A

REGISTER NO : 1911260245

DEPARTMENT : B.TECH COMPUTER SCIENCE ENGINEERING

ACADEMIC YEAR : 2026

FACULTY NAME : KUMARAN G

NATURE OF ASSIGNMENT : INDIVIDUAL

1. Introduction

A vibration sensor in the plant sends 16-bit readings over a noisy radio link. If a reading is corrupted, a retransmission takes **40 ms**. The engineer must decide whether the receiver should correct errors by itself using **Forward Error Correction (FEC)** or detect the error and ask the sender to repeat the reading using **Automatic Repeat Request (ARQ)**. This report answers all parts of Task 8: Hamming check bits for 16 data bits, the minimum distance of a given code, a worked Hamming(7,4) correction, a two-bit miscorrection demonstration, an FEC-versus-ARQ comparison, and a numerical alternative-concept analysis at a bit-error rate of 0.001.

All numbers below were calculated by hand and then independently re-checked with a short program. No experimental or simulated measurements are claimed; every result is analytical.

2. Hamming Check-Bit Calculation (16-bit word)

For a single-error-correcting Hamming code with **m** data bits and **r** check bits, the condition is **2^r ≥ m + r + 1**. Here m = 16. We test increasing values of r:

r	2^r	m + r + 1	Condition 2^r ≥ m + r + 1
3	8	16 + 3 + 1 = 20	8 ≥ 20 → False
4	16	16 + 4 + 1 = 21	16 ≥ 21 → False
5	32	16 + 5 + 1 = 22	32 ≥ 22 → **True (smallest r)**

```
Check bits           r = 5
Total codeword      n = m + r = 16 + 5 = 21 bits   → Hamming(21,16)
Overhead (of data)  = r / m = 5 / 16 = 0.3125 = 31.25 %
Overhead (of code)  = r / n = 5 / 21 = 0.2381 = 23.81 %
```

Denominator used: the headline overhead of **31.25 %** is calculated against the **16 data bits** (extra bits per data bit). As a share of the complete 21-bit codeword, the check bits are 23.81 %. Final answer: **5 check bits, 21-bit codeword, 31.25 % overhead on the data (23.81 % of the codeword).**

3. Minimum Hamming Distance of the Code

Code C = {00000, 01011, 10101, 11110}. The Hamming distance between two codewords is the number of bit positions where they differ, i.e. the number of 1s in their XOR. With 4 codewords there are C(4,2) = 6 pairs.

Pair	Codeword A	Codeword B	A XOR B	Differing bits (d)
1	00000	01011	01011	3
2	00000	10101	10101	3
3	00000	11110	11110	4
4	01011	10101	11110	4
5	01011	11110	10101	3
6	10101	11110	01011	3

```
Distances found : 3, 3, 4, 4, 3, 3
d_min = 3
Errors detected  = d_min − 1            = 3 − 1 = 2
Errors corrected = floor((d_min − 1)/2) = floor(2/2) = 1
```

Final answer: **minimum Hamming distance = 3; the code detects up to 2 errors and corrects up to 1 error.**

4. Hamming(7,4) Error Correction

Convention: even parity, positions numbered **1 to 7 from left to right**, check bits at positions **1, 2 and 4** (powers of two), data bits at positions 3, 5, 6, 7. Each check bit covers the positions whose binary index contains that power of two: P1 → {1,3,5,7}, P2 → {2,3,6,7}, P4 → {4,5,6,7}.

4.1 Bit positions

Position	1	2	3	4	5	6	7
Role	P1	P2	D1	P4	D2	D3	D4
Original 0110011	0	1	1	0	0	1	1
Received 0110111	0	1	1	0	1	1	1

(Check: the original codeword has even parity in all three groups, so its syndrome is 000.)

4.2 Parity checks on the received word 0110111

```
Received bits: pos1=0 pos2=1 pos3=1 pos4=0 pos5=1 pos6=1 pos7=1

C1 = bits(1,3,5,7) = 0 ⊕ 1 ⊕ 1 ⊕ 1 = 3 ones → odd  → C1 = 1
C2 = bits(2,3,6,7) = 1 ⊕ 1 ⊕ 1 ⊕ 1 = 4 ones → even → C2 = 0
C4 = bits(4,5,6,7) = 0 ⊕ 1 ⊕ 1 ⊕ 1 = 3 ones → odd  → C4 = 1
```

4.3 Syndrome and correction

```
Syndrome (C4 C2 C1) = 1 0 1 (binary) = 1×4 + 0×2 + 1×1 = 5 (decimal)
Error position   = 5   (the 5th bit from the left)

Received   : 0 1 1 0 1 1 1
Flip bit 5 : 0 1 1 0 0 1 1
Corrected codeword = 0110011  (identical to the original)

Data bits at positions 3,5,6,7 = 1, 0, 1, 1
Extracted 4-bit data = 1011
```

Final answer: **bit 5 was in error; corrected codeword 0110011; corrected data 1011.**

5. Two-Bit Error and Miscorrection Demonstration

Choose to flip **bit 1 and bit 2** of the original codeword 0110011 (a two-bit error).

```
Position        :  1 2 3 4 5 6 7
Original       :  0 1 1 0 0 1 1
Error pattern  :  1 1 0 0 0 0 0   (bits 1 and 2 flipped)
Received        :  1 0 1 0 0 1 1   = 1010011
```

5.1 Syndrome of 1010011

```
C1 = bits(1,3,5,7) = 1 ⊕ 1 ⊕ 0 ⊕ 1 = 3 ones → odd  → 1
C2 = bits(2,3,6,7) = 0 ⊕ 1 ⊕ 1 ⊕ 1 = 3 ones → odd  → 1
C4 = bits(4,5,6,7) = 0 ⊕ 0 ⊕ 1 ⊕ 1 = 2 ones → even → 0

Syndrome (C4 C2 C1) = 0 1 1 = 0×4 + 1×2 + 1×1 = 3
```

5.2 The decoder's wrong correction

```
Decoder assumes ONE error at position 3 and flips bit 3:
Received        : 1 0 1 0 0 1 1
Decoder output  : 1 0 0 0 0 1 1   = 1000011

Original sent   : 0 1 1 0 0 1 1   = 0110011
Decoder output  : 1 0 0 0 0 1 1   = 1000011
Bits that differ: positions 1, 2 and 3  → 3 wrong bits
Data extracted  : positions 3,5,6,7 = 0,0,1,1 = 0011  (original was 1011)
```

The decoder output **1000011 ≠ 0110011**. The receiver silently delivers corrupted data (0011 instead of 1011), and the error is now worse than before: the original two errors have become three.

5.3 Why Hamming(7,4) corrects one error but may miscorrect two

The syndrome of a single error at position i equals the binary number i, so every non-zero syndrome 1 to 7 points to exactly one position. Because the syndrome is the XOR of the position numbers of all flipped bits, two errors at positions i and j give the syndrome **i XOR j**, which is just the position number of some other bit k. Here 1 XOR 2 = 001 XOR 010 = 011 = 3, so the decoder blames bit 3. Hamming(7,4) has minimum distance **d_min = 3**: it can correct floor((3−1)/2) = 1 error, and it can detect 2 errors only if it is used for detection and not correction. A two-error pattern lies at distance 1 from a different valid codeword, so correcting mode picks that wrong codeword. An extra overall parity bit (SECDED, Hamming(8,4)) raises d_min to 4 and lets the receiver detect, but not correct, double errors.

6. FEC versus ARQ

Forward Error Correction (FEC) adds redundant check bits to every transmission so that the receiver can locate and correct certain errors without asking for anything again. **Automatic Repeat Request (ARQ)** only needs enough redundancy to detect an error (for example a parity bit, checksum or CRC); when the receiver finds an error it requests retransmission from the sender.

Criterion	FEC (e.g. Hamming)	ARQ (detect + retransmit)
Working principle	Redundant check bits let the receiver correct limited errors by itself.	Check bits only detect errors; the receiver asks the sender to resend the frame (ACK/NAK, timeout).
Transmission overhead	Constant: check bits on every reading, even when there is no error (5 bits per 16 bits for Hamming(21,16)).	Small fixed detection code, plus extra transmissions only when errors occur; grows with error rate.
Delay	No retransmission delay for correctable errors; decoding time is very small.	Each error adds at least one retransmission, here 40 ms, plus ACK/NAK handling; delay varies (jitter).
Reliability	Fails when errors exceed the code's capability (e.g. 2 errors in Hamming) and may miscorrect silently.	Very high: with a good CRC, bad data is almost always detected and repeated until correct.
Suitable applications	Real-time or one-way links, long-delay links, broadcast: video streaming, satellite, 5G radio, storage media.	Links with low error rate and short round trip where delay is tolerable: file transfer, TCP, HDLC/Ethernet-type links.

6.1 Which method suits this noisy radio link?

FEC (or FEC with a light ARQ backup) suits the vibration-sensor link better. Vibration readings are time-sensitive, and each retransmission costs 40 ms, which is long compared with a sensor sampling period. On a noisy radio channel errors are frequent, so ARQ would repeatedly trigger these 40 ms delays and add jitter, and each repetition also uses scarce radio airtime and battery. FEC adds a fixed 5 bits per reading but removes the delay for the most common case, a single-bit error. Section 7 puts numbers on this trade-off.

6.2 Why video streaming uses FEC

Streaming video is delay-sensitive and often one-to-many. Waiting for a retransmission would make the frame arrive after its playback deadline, so it would be useless anyway, and a server cannot answer thousands of viewers' repeat requests. FEC lets the receiver rebuild lost or corrupted data from redundancy already received, keeping playback smooth with only a small, constant bandwidth overhead.

6.3 Why 5G uses FEC (and hybrid ARQ)

A 5G radio channel suffers fading, interference and noise, giving high raw bit-error rates, and the target latency is very low (a few milliseconds for URLLC). Strong FEC codes (LDPC for data channels, polar codes for control channels) correct most errors at the first reception so the link meets its delay and throughput targets. 5G may also

use **Hybrid ARQ (HARQ)**, which combines FEC with fast retransmission: if FEC cannot correct a block, the receiver requests a retransmission and combines it with the earlier copy, so the combined result is more reliable than either alone.

7. Alternative Concept Analysis (BER = 0.001)

Given: bit-error rate p = 1 in 1,000 = **0.001**; data reading = **16 bits**; retransmission time = **40 ms**. The two approaches are compared as Hamming FEC versus ARQ.

7.1 A. Hamming FEC

```
Check bits r = 5   (from Section 2, 2^5 = 32 ≥ 22)
Total bits sent per reading = 16 + 5 = 21 bits  (Hamming(21,16))
Extra retransmission delay for a single-bit error = 0 ms
```

Hamming(21,16) has d_min = 3, so it **corrects any single-bit error** in the 21-bit codeword but **cannot guarantee correction of two or more bit errors**; as shown in Section 5, a multi-bit error can be miscorrected silently.

7.2 B. ARQ (independent bit errors, 16-bit reading)

```
P(success)      = (1 − 0.001)^16 = 0.999^16
                = 0.98412    (0.984119...)
P(error)        = 1 − P(success)
                = 1 − 0.98412
                = 0.01588   (1.588 %)
E[transmissions]= 1 / P(success)
                = 1 / 0.98412
                = 1.01614
Avg data bits   = 16 / P(success)
                = 16 × 1.01614
                = 16.258 bits
E[retransmits]  = (1 − P(success)) / P(success)
                = 0.01588 / 0.98412
                = 0.016137
Avg extra delay = E[retransmits] × 40 ms
                = 0.016137 × 40 ms
                = 0.6455 ms ≈ 0.65 ms
```

Assumptions: these are **expected (average) values** over many readings, with independent bit errors. **ACK/control overhead and the initial transmission time are excluded** because the assignment does not specify them. The ARQ figures also assume the error-detection code overhead is not counted, since the task gives only the 16 data bits.

7.3 C. Comparison

Criterion	Hamming FEC	ARQ
Bits sent per successful reading	**21 bits** (always)	**16.258 bits** on average (16 / 0.98412)
Average additional retransmission waiting time	**0 ms** for single-bit errors	**0.65 ms** on average (0.016137 × 40 ms), but **40 ms** for each individual retransmission (1.6 % of readings)
Error-handling capability	Corrects 1 bit error per codeword; multi-bit errors may be missed or miscorrected	Needs only error detection; repeats until a correct copy arrives, so multi-bit errors are handled
Main advantage	Predictable, fixed latency; no feedback channel needed	Lower bit overhead and higher reliability at this low error rate
Main limitation	Always 5 extra bits (31.25 % of data); fails on 2+ errors	Delay spikes of 40 ms and jitter; needs a return path; overhead rises sharply if BER increases

7.4 Justified choice for the sensor radio link

At the stated BER of 0.001 only about **1.6 %** of 16-bit readings are corrupted, so ARQ is very cheap on average: **16.26 bits** and only **0.65 ms** of expected extra delay, compared with a permanent **21 bits** for Hamming. However,

the average hides the worst case: one reading in about 63 needs a **40 ms** retransmission. For a vibration sensor, where timely and regularly spaced data matter and where a noisy radio link may have a far worse error rate than 0.001 (especially near welding machines), I recommend **FEC (Hamming-type correction) as the primary method, combined with a CRC-based ARQ as a fallback for blocks the FEC cannot fix** (the hybrid approach used by 5G HARQ). If the link were quiet, delay-tolerant and battery-limited, plain ARQ would be preferable because it sends fewer bits on average.

8. Conclusion

- A 16-bit word needs **5 Hamming check bits** (2^5 = 32 ≥ 22), giving a **21-bit** codeword with **31.25 %** overhead on the data (23.81 % of the codeword).
- The code {00000, 01011, 10101, 11110} has **d_min = 3**, so it **detects 2** and **corrects 1** error.
- Received 0110111 gives syndrome 101 = 5, so bit 5 is corrected, producing **0110011** with data **1011**.
- Flipping bits 1 and 2 gives syndrome 011 = 3; the decoder wrongly flips bit 3 and outputs **1000011**, which differs from 0110011 in three positions.
- At BER 0.001, ARQ needs on average **16.258 bits** and **0.65 ms** extra delay versus **21 bits** and **0 ms** for Hamming FEC; because of the 40 ms retransmission spikes and the noisy link, FEC with ARQ fallback is the recommended design.